UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 8 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The addressing technique illustrated in Figure 2 is direct addressing (also called absolute addressing). The instruction's address field A holds the actual/effective address of the operand, so the operand is fetched directly from memory location A — no further address calculation is performed.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A from the instruction and places it on the address bus to access main memory. The operand stored at memory location A is read into a register (or used by the ALU) and the operation named by the opcode is executed. In symbolic form: EA = A, then Operand = Memory[EA]. Because the effective address is obtained in a single memory reference after the instruction fetch, direct addressing is simple and fast, but the address field must be wide enough to hold a full memory address.

(c) Advantage: No additional address calculation or extra memory access is needed to determine the effective address — the operand is reached in one memory access, making it simple and fast. Disadvantage: The address field must be large enough to hold a complete memory address, which limits the range of addressable memory (and the number of bits left for the opcode); it also offers no flexibility such as relocation or indexing.

Unofficial marking guide:

- *1 mark: Correctly labels the technique as direct (absolute) addressing.*
- *1 mark: States that the address field A is the effective address of the operand (EA = A).*
- *1 mark: Explains the fetch/execute step — address A is sent to memory and the operand at Memory[A] is retrieved and used by the opcode operation.*
- *1 mark: Gives ONE valid advantage (e.g., no extra address calculation, single memory access, simple/fast).*
- *1 mark: Gives ONE valid disadvantage (e.g., address field must be full width, limiting addressable memory/opcode bits; no relocation/indexing flexibility).*

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, pointing to Memory: Operand). This is an unofficial model answer, not an official marking scheme.

QUESTION A2 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The question asks for completion of a taxonomy of parallel processor architecture with appropriate examples. The standard taxonomy (Flynn-style / shared-memory vs message-passing) is filled in as follows, with one example per category:

[Taxonomy completion] Parallel processor architectures divide into two main branches: (i) Shared-memory multiprocessors (tightly coupled), where all processors access a common memory, and (ii) Message-passing multicomputers (loosely coupled), where each processor has its own private memory and processors communicate by passing messages over an interconnection network.

[Shared-memory branch] Shared-memory multiprocessors subdivide into: (a) Uniform Memory Access (UMA) — all processors have equal access time to all memory modules, e.g. a symmetric multiprocessor (SMP) such as a dual/quad-core Intel Xeon server; (b) Non-Uniform Memory Access (NUMA) — memory access time depends on which memory module is addressed, e.g. a large-scale cache-coherent NUMA (ccNUMA) server such as SGI Origin; (c) Cache-Only Memory Architecture (COMA) — memory is treated as a pool of caches, e.g. the Kendall Square Research KSR-1.

[Message-passing branch] Message-passing multicomputers subdivide into: (a) Massively Parallel Processors (MPP) — many processors with private memories connected by a dedicated high-speed network, e.g. Intel Paragon or Cray T3D; (b) Clusters / Workstations-of-workstations (COWs) — commodity computers connected by a LAN, e.g. a Beowulf cluster; (c) Distributed shared-memory systems built on message passing, e.g. a cluster running a DSM/software DSM layer.

© Hak Cipta Universiti Teknologi MARA

[Example requirement] Each leaf of the taxonomy must be paired with a named, real example as shown above; the example should match the defining property of that category (equal access time for UMA, private memory plus network for MPP, commodity nodes for clusters).

Unofficial marking guide:

• 1 mark: correct top-level division into shared-memory (tightly coupled) and message-passing (loosely coupled) branches

• 1 mark: correct sub-categories under shared memory (UMA / NUMA / COMA)

• 1 mark: correct sub-categories under message passing (MPP / cluster / COW)

• 1 mark: a valid, named example for each shared-memory sub-category

• 1 mark: a valid, named example for each message-passing sub-category

Model confidence: medium.

Model note: The taxonomy figure (CSC429-2016-06-D1) is only available as extracted text, so the exact branch labels and number of leaves in the original diagram cannot be verified. The answer assumes the standard Flynn/shared-vs-message-passing taxonomy used in CSC429 notes; if the figure uses different labels (e.g. SIMD/MIMD or UMA/NUMA/COMA only), map the examples accordingly. Examples are illustrative and any correct named system should be accepted.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Answer: -6.7510 = 0xC0D80000 (sign=1, exponent=10000001, fraction=10110000000000000000)

• 1 mark for the sign bit.

• 1 mark for the biased exponent.

• 1 mark for the 23-bit fraction.

• 1 mark for the correct 32-bit grouping.

• 1 mark for the final hexadecimal value.

QUESTION A4 (5 marks)

Topic: Computer generations and history

Source: CSC429 (16).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two prominent first-generation machines inspired by John von Neumann's stored-program concept were the EDVAC (Electronic Discrete Variable Automatic Computer) and the EDSAC (Electronic Delay Storage Automatic Calculator). Other acceptable first-generation stored-program machines include the IAS machine, UNIVAC I and Manchester Baby; the two most commonly cited in CSC429 material are EDVAC and EDSAC.

(b) The machine developed by John von Neumann himself is the IAS machine (Institute for Advanced Study computer), sometimes called the von Neumann machine.

(c) The IAS machine was a stored-program electronic digital computer. Its function was to hold both program instructions and data in the same main memory and execute instructions sequentially under the control of a central processing unit, following the fetch–decode–execute cycle. This demonstrated the von Neumann (stored-program) architecture: instructions are fetched from memory, decoded and executed one at a time, with the program counter controlling the sequence, and results written back to memory. It was used for scientific and numerical computation and became the prototype for later general-purpose computers.

Unofficial marking guide:

• 1 mark: naming EDVAC (or another valid first-generation stored-program machine)

• 1 mark: naming EDSAC (or a second valid first-generation stored-program machine)

• 1 mark: correctly naming the IAS machine / von Neumann machine

• 1 mark: stating that program and data are stored together in memory (stored-program concept)

• 1 mark: explaining sequential fetch–decode–execute execution under CPU control / general-purpose scientific computation

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Accept other valid first-generation stored-program machines (e.g. UNIVAC I, Manchester Baby) for part (a) if the lecturer's list differs.

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

© Hak Cipta Universiti Teknologi MARA

Source: CSC429 (17).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of bus timing are synchronous timing and asynchronous timing. Synchronous bus timing uses a common clock signal to coordinate all bus transactions; events occur at fixed clock edges, so the bus protocol is simple and fast for devices that can operate at the clock rate, but it forces all devices to run at the speed of the slowest device and becomes inefficient over long distances due to clock skew. Asynchronous bus timing does not use a common clock; instead, it uses handshaking signals (e.g., request/acknowledge) so that a master waits for a slave to respond, allowing devices of different speeds to communicate and making the bus robust over long distances, at the cost of extra control lines and more complex protocol overhead.

(b) Three CPU components: (1) Control Unit (CU): fetches and decodes instructions, generates control signals to coordinate the datapath, ALU, registers, and memory/I/O; it directs the operation of the processor. (2) Arithmetic Logic Unit (ALU): performs arithmetic operations (add, subtract, etc.) and logical operations (AND, OR, NOT, shifts) on data held in registers; it is the computational core. (3) Registers: small, fast storage inside the CPU, including the Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Data Register (MDR), and general-purpose registers; they hold operands, addresses, and intermediate results during instruction execution.

Unofficial marking guide:

• 1 mark: correct identification of synchronous bus timing
• 1 mark: explanation of synchronous timing (common clock, fixed events, advantages/limitations)
• 1 mark: correct identification of asynchronous bus timing
• 1 mark: explanation of asynchronous timing (handshaking, no common clock, advantages/limitations)
• 2 marks: description of Control Unit (function: fetch/decode, control signals, coordination)
• 2 marks: description of ALU (arithmetic and logical operations)
• 2 marks: description of Registers (fast storage, examples such as PC, IR, MAR, MDR, general-purpose)

Model confidence: high.

Model note: Question text appears complete; no OCR corrections needed. The marking guide allocates 4 marks for part (a) and 6 marks for part (b), totaling 10 marks.

QUESTION A6 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (6).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The size of a bus is not determined solely by the word size. The bus width (number of data lines) is typically equal to the word size for a data bus, so if the word size is 64 bits, the data bus size would be 64 bits. However, the address bus size depends on the addressable memory, and the control bus size depends on the control signals. Therefore, the bus size (data bus) is 64 bits.

(b) The two types of bus are: (1) Data bus, (2) Address bus. (Alternatively, control bus is also a type; but the two main types are data and address buses.)

(c) The bus is located between the CPU and memory as a set of parallel wires (data, address, control) that connect the CPU's memory interface to the memory unit. In a simple diagram: CPU is connected to the system bus, which in turn connects to memory. The bus acts as a shared communication pathway.

(d) System interconnection refers to the overall mechanism (e.g., system bus) that connects major components (CPU, memory, I/O) and allows communication among them. CPU interconnection refers specifically to the internal buses or pathways within the CPU (e.g., between registers, ALU, control unit) or the connection between CPU and other components via the system bus. The key difference is scope: system interconnection is broader, covering all system components, while CPU interconnection is narrower, focusing on the CPU's internal or dedicated connections.

(e) Drawback of synchronous bus timing: all devices must operate at the same clock rate, so the bus speed is limited by the slowest device, reducing overall performance. Drawback of asynchronous bus timing: it requires handshaking overhead (request/acknowledge signals), which increases complexity and can slow down transfers due to propagation delays.

Unofficial marking guide:

• 1 mark: (a) Correctly states that the data bus size is 64 bits (or that bus size equals word size for data bus).
• 2 marks: (b) Lists two correct bus types (e.g., data bus and address bus; or data bus and control bus; or address bus and control bus). 1 mark each.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: (c) Provides a correct illustration (diagram or description) showing the bus between CPU and memory.
• 2 marks: (d) Correctly differentiates system interconnection and CPU interconnection. 1 mark for defining each or 2 marks for clear contrast.
• 4 marks: (e) Discusses one drawback each for synchronous and asynchronous bus timing. 2 marks per drawback (1 mark for identifying drawback, 1 mark for explanation).
Model confidence: medium.
Model note: Assumption: 'bus size' in (a) is interpreted as data bus width. The question may expect '64 bits' as answer. For (b), the two types could be data and address buses, or synchronous and asynchronous buses; the answer given uses data and address buses as they are fundamental. For (e), the two types of bus timing are synchronous and asynchronous; the drawbacks are standard. The illustration in (c) is described textually; a diagram would be needed for full marks.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts
Source: CSC429 (7).pdf, page(s) 5
Unofficial model answer — generated by an AI model, not an official UTM marking scheme.
(a) Differentiate between interrupt-driven I/O and programmed I/O [4 marks]
In programmed I/O (polled I/O), the CPU issues an I/O command to a device and then repeatedly checks the device's status register in a polling loop until the device is ready, after which the CPU transfers the data itself. The CPU is fully occupied while waiting, so no other useful work is done during the wait, and the transfer is synchronised by the program rather than by hardware. In interrupt-driven I/O, the CPU issues the command and continues executing other instructions; when the device is ready it raises an interrupt request, the CPU suspends the current program, saves its context, and branches to an interrupt service routine (ISR) that performs the data transfer, then returns to the interrupted program. Thus programmed I/O wastes CPU time in busy-waiting and has no hardware handshaking overhead, whereas interrupt-driven I/O frees the CPU to do useful work but incurs interrupt overhead (context save/restore, ISR entry) and requires interrupt hardware and priority handling.

(b) One benefit of programmed I/O [2 marks]
Programmed I/O requires no interrupt hardware or ISR/context-switching mechanism, so it is simple and cheap to implement and needs no special processor support; it is also fully predictable and deterministic, which suits simple, dedicated systems (e.g. a single sensor or a slow device) where the CPU has nothing else to do while waiting.

(c) Two disadvantages of multiple interrupts [4 marks]
1. Interrupt overhead and reduced throughput: each interrupt forces the CPU to save the current context, enter the ISR, and restore context on return; with many interrupts arriving, a large fraction of CPU time is spent on this overhead rather than on useful work, lowering effective throughput.
2. Priority and nesting problems: when several interrupts occur together, the CPU must decide which to service first, and a low-priority ISR can be pre-empted by a higher-priority one, causing unpredictable delays, possible loss of a pending interrupt if it is not latched, and difficulty in guaranteeing response time to critical devices. (Related risks include re-entrancy bugs and stack growth from nested ISRs.)

Unofficial marking guide:
• 1 mark: correct definition of programmed I/O as CPU polling/busy-waiting on device status
• 1 mark: correct definition of interrupt-driven I/O as device signalling the CPU via an interrupt request and ISR handling
• 1 mark: contrast on CPU utilisation (busy-wait vs CPU free to do other work)
• 1 mark: contrast on overhead/hardware requirements (no interrupt hardware vs context save/restore and priority handling)
• 2 marks: one valid benefit of programmed I/O (e.g. simple/cheap, no interrupt hardware, deterministic) with a brief justification
• 2 marks: first disadvantage of multiple interrupts (e.g. context-switch/ISR overhead reducing throughput) explained
• 2 marks: second disadvantage of multiple interrupts (e.g. priority/nesting causing delays or lost interrupts) explained
Model confidence: high.
Model note: OCR errors '1/0' read as 'I/O'. The question asks for TWO disadvantages of multiple interrupts; the answer gives two distinct points, each worth 2 marks. No figure or table is required.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts
Source: CSC429 (11).pdf, page(s) 3

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The non-maskable interrupt (NMI) is message (b): "A hardware failure has been found. Please restart your computer now." Justification: an NMI is reserved for catastrophic events such as a hardware failure or memory error; it cannot be disabled (masked) by software, so the CPU must service it immediately. Message (a), the upgrade offer, is an ordinary maskable interrupt from an I/O device (the web/booking service); the user can ignore or defer it, so it can be masked.

(b) Maskable interrupt: can be disabled (masked) by the CPU, usually via an interrupt-enable flag or interrupt mask register; it is used for normal I/O events and may be deferred. Non-maskable interrupt: cannot be disabled by software; it is reserved for urgent/critical events such as hardware failure and is always serviced.

(c) Common way to handle multiple interrupts: use interrupt priority. Each interrupt source is assigned a priority; when several interrupts arrive, the CPU services the highest-priority one first. If a higher-priority interrupt arrives while a lower-priority one is being serviced, the CPU can pre-empt (interrupt) the lower-priority handler and service the higher-priority one first; lower-priority requests wait. In the scenario, the hardware-failure NMI (message b) has higher priority than the upgrade request (message a), so the NMI is handled first and the upgrade request is handled afterwards or masked/deferred.

Unofficial marking guide:

- *1 mark: identifying message (b) as the non-maskable interrupt*
- *1 mark: stating that an NMI is for critical/hardware failure events*
- *1 mark: stating that an NMI cannot be masked/disabled by software*
- *1 mark: contrasting message (a) as a maskable/normal I/O interrupt that can be deferred*
- *1 mark: defining maskable interrupt as one that can be disabled/masked by the CPU*
- *1 mark: defining non-maskable interrupt as one that cannot be disabled and is for urgent events*
- *1 mark: stating that multiple interrupts are handled by priority*
- *1 mark: explaining that the highest-priority interrupt is serviced first*
- *1 mark: explaining pre-emption/nesting when a higher-priority interrupt arrives during a lower-priority handler*
- *1 mark: applying priority to the scenario (NMI/hardware failure handled before the upgrade request)*

Model confidence: high.

Model note: Figure 1 is represented only by its extracted text; the answer assumes (a) is the upgrade message and (b) is the hardware-failure message as labelled in the question. The question is worth 10 marks; the marking guide allocates 4 + 2 + 4 marks across subparts (a), (b) and (c).

QUESTION A9 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (20).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Bit storage in SRAM and DRAM [2 marks]

SRAM stores each bit in a bistable flip-flop/latch built from four to six transistors (typically two cross-coupled inverters plus two access transistors). As long as power is supplied, the cell holds its state indefinitely without refresh; the bit is read by sensing the state driven onto the bit lines. DRAM stores each bit as charge on a small capacitor (one transistor plus one capacitor per cell). A charged capacitor represents logic 1 and a discharged capacitor logic 0 (or the reverse convention). Because the capacitor leaks, the charge must be periodically refreshed (every few milliseconds) to retain the data. (Note: 'DFiAM' in the question is taken to be an OCR error for DRAM.)

(b) Why SRAM is faster than DRAM [4 marks]

1. Sense mechanism: SRAM cells actively drive the bit lines through transistors, so a small voltage swing is detected quickly by a differential sense amplifier. DRAM must detect a much smaller charge sharing on highly capacitive bit lines, which takes longer.
2. No refresh/restore overhead: SRAM needs no refresh cycles and no write-back after a read. A DRAM read is destructive (the capacitor charge is shared with the bit line), so the value must be sensed and then restored, adding delay.
3. Access path: SRAM is built with fast logic-compatible transistors and short interconnect, giving low access latency (a few nanoseconds). DRAM uses a single transistor per cell with a large capacitance and longer row/column access paths, giving higher latency (tens of nanoseconds).
4. No precharge/activate sequencing: DRAM requires row activation, precharge and column access steps (RAS/CAS style timing), whereas SRAM access is a simple address-decode-and-sense operation.

(c) Comparison of SRAM and DRAM: cost and power [4 marks]

© Hak Cipta Universiti Teknologi MARA

Cost: SRAM uses 4–6 transistors per bit, so its cell area is much larger and its cost per bit is high; it is used for small, fast caches. DRAM uses 1 transistor + 1 capacitor per bit, giving much higher density and a much lower cost per bit; it is used for main memory.
Power: SRAM consumes static (leakage) power while powered but has low dynamic power per access and no refresh power. DRAM has low static leakage per cell but requires continuous refresh power and consumes dynamic power on every activate/precharge/read/write; overall DRAM has higher standby/refresh power but lower cost per bit. In summary, SRAM is faster and more expensive per bit with higher leakage, while DRAM is denser, cheaper per bit, but slower and needs refresh power.

Unofficial marking guide:

• 1 mark: SRAM stores a bit in a bistable flip-flop/latch (cross-coupled inverters, 4–6 transistors) and holds it while powered.
• 1 mark: DRAM stores a bit as charge on a capacitor (1T1C) and needs periodic refresh.
• 1 mark: SRAM bit lines are actively driven and sensed quickly by a differential sense amplifier.
• 1 mark: DRAM sensing involves small charge sharing on capacitive bit lines, which is slower.
• 1 mark: DRAM reads are destructive and require restore, and DRAM needs refresh/precharge/activate sequencing, adding delay.
• 1 mark: SRAM has no refresh and a simpler address-decode-and-sense access path, so it is faster.
• 1 mark: Cost — SRAM has 4–6 transistors/bit, large cell area, high cost per bit; used for caches.
• 1 mark: Cost — DRAM has 1T1C/bit, high density, low cost per bit; used for main memory.
• 1 mark: Power — SRAM has static leakage but low dynamic power and no refresh power.
• 1 mark: Power — DRAM needs refresh power and dynamic activate/precharge power, though low leakage per cell.
Model confidence: high.
Model note: The question text says 'DFiAM'; this is treated as an OCR error for DRAM. No figure or table is required.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: 137710 = 56116

• 1 mark for repeated division by 16.
• 1 mark for correct remainders.
• 1 mark for reading remainders in reverse order.
• 1 mark for the correct hexadecimal digits.
• 1 mark for the base subscript.

b) Answer: 4B816 = 120810

• 1 mark for place values 16^2, 16^1 and 16^0.
• 1 mark for correct digit values.
• 1 mark for correct products.
• 1 mark for correct addition.
• 1 mark for the final decimal value.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA